

Reviewer Pack — Minimal Hyperbolic Volume of Graphs and Resolution Proof Width...

Entry 7d70057c2c91 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-07 08:24:05 UTC

Minimal Hyperbolic Volume of Graphs and Resolution Proof Width Inequality

Entry ID: 7d70057c2c91 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-07 08:24:05 UTC

1. Conjecture

Field A (mathematical branch): Geometric Group Theory (Hyperbolic Geometry) **Field B** (complexity object): Boolean Satisfiability (Resolution Proof Complexity)

Statement:

For every d -regular graph G with n vertices, the minimal hyperbolic volume $V(G)$ of its universal covering space is linearly correlated with its resolution proof width $w(\varphi_G)$, such that $V(G) = \Omega(w(\varphi_G))$.

Rationale (proposer's reasoning):

Hyperbolic geometry provides a rich mathematical framework for studying geometric properties of graphs. The minimal hyperbolic volume of a graph's universal covering space has been used in other areas of mathematics, but its application to resolution proof complexity is novel. A larger hyperbolic volume might indicate a more complex structure that the resolution process needs to navigate, suggesting a lower bound on proof width.

Taxonomy category: geometric_group_theory_hyperbolic_geometry (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 57c82dd5f6943c4a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For the conjecture on minimal hyperbolic volume of graphs and resolution proof width inequality, support is achieved if the mean minimal hyperbolic volume $V(G)$ for all d -regular graphs G with n vertices is greater than or equal to the mean resolution proof width $w(\varphi_G)$ across at least 30 random seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal hyperbolic volume" AND "graph universal covering" AND "resolution proof complexity" - "d-regular graph" AND "hyperbolic geometry" AND "Boolean satisfiability" - "geometric group theory" AND "hyperbolic geometry" AND "resolution proof width inequality"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_d_regular_graph(n, d):
        if (n * d) % 2 != 0:
            return None
        graph = [[] for _ in range(n)]
        edges_used = set()
        for i in range(n):
            for j in range(d // 2):
                while True:
                    neighbor = random.randint(0, n - 1)
                    if neighbor == i or (i, neighbor) in edges_used or (neighbor, i) in edges_used:
                        continue
                    graph[i].append(neighbor)
                    graph[neighbor].append(i)
                    edges_used.add((i, neighbor))
                break
        return graph

    def gaussian_elimination(matrix):
        n = len(matrix)
        for i in range(n):
            max_row = i
            for j in range(i + 1, n):
                if abs(matrix[j][i]) > abs(matrix[max_row][i]):
                    max_row = j
            matrix[i], matrix[max_row] = matrix[max_row], matrix[i]
            factor = matrix[i][i]
```

```

    for j in range(i, n):
        matrix[i][j] /= factor
    for j in range(n):
        if j != i:
            factor = matrix[j][i]
            for k in range(i, n):
                matrix[j][k] -= factor * matrix[i][k]
    return matrix

def determinant(matrix):
    n = len(matrix)
    det = 1
    for i in range(n):
        det *= matrix[i][i]
    return det

def hyperbolic_volume(graph):
    n = len(graph)
    adjacency_matrix = [[0] * n for _ in range(n)]
    for u, v in graph:
        adjacency_matrix[u][v] = 1
        adjacency_matrix[v][u] = 1

    laplacian_matrix = [[0] * n for _ in range(n)]
    for i in range(n):
        degree = sum(adjacency_matrix[i])
        laplacian_matrix[i][i] = -degree
        for j in range(i + 1, n):
            laplacian_matrix[i][j] = adjacency_matrix[i][j]
            laplacian_matrix[j][i] = adjacency_matrix[j][i]

    return abs(determinant(laplacian_matrix))

def resolution_proof_width(graph):
    n = len(graph)
    variables = set()
    for u, v in graph:
        variables.add(u)
        variables.add(v)

    clauses = []
    for u, v in graph:
        clauses.append([u, -v])
        clauses.append([-u, v])

def is_satisfiable(clauses):
    assignment = {}
    stack = []
    for var in variables:
        if var not in assignment and -var not in assignment:
            stack.append((var, True))

    while stack:
        var, value = stack.pop()
        assignment[var] = value
        for clause in clauses:

```

```

        if value == True and var in clause:
            continue
        if value == False and -var in clause:
            continue
        if all(x not in assignment or (assignment[x] == True and x in clause) or (assi
            return False
        new_var = next((x for x in clause if x != var and -x != var), None)
        if new_var is not None:
            stack.append((-new_var, not value))
    return True

width = 0
while len(clauses) > 0:
    max_clause_length = max(len(clause) for clause in clauses)
    if max_clause_length <= width:
        break
    width += 1
    new_clauses = []
    for clause in clauses:
        if len(clause) == max_clause_length:
            new_clauses.extend([c for c in clause if len(c) < max_clause_length])
            break
        new_clauses.append(clause)
    clauses = new_clauses

return width

n_values = [5, 10, 15, 20, 30, 40]
total_v = 0
total_w = 0
instances_tested = 0
n_max = 0

for n in n_values:
    d = random.randint(3, min(n - 1, 8))
    graph = generate_d_regular_graph(n, d)
    if graph is None:
        continue

    v = hyperbolic_volume(graph)
    w = resolution_proof_width(graph)

    total_v += v
    total_w += w
    instances_tested += 1
    n_max = max(n_max, n)

mean_v = total_v / instances_tested if instances_tested > 0 else 0
mean_w = total_w / instances_tested if instances_tested > 0 else 0

conjecture_holds = mean_v >= mean_w
counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": "Hyperbolic Volume vs Resolution Proof Width",
    "metric_value": mean_v,

```

```

        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19,

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_v = sum(r["metric_value"] for r in results) / len(results) if len(results) > 0 else 0
    mean_w = sum(r["instances_tested"] * r["metric_value"] for r in results) / sum(r["instances_tested"] for r in results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_v} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_v} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it did not complete within the allotted time frame. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15356
2	propose	ollama_remote	glm4:latest	0	0	13806
3	preregistration	ollama_remote	glm4:latest	0	0	22261
4	novelty	ollama_remote	glm4:latest	0	0	9457
5	novelty	ollama_remote	glm4:latest	0	0	9556
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	112382
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10788
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8195
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18871
10	judge	ollama_remote	glm4:latest	0	0	13234

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 233908 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in *research/audit/7d70057c2c91.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/7d70057c2c91.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/7d70057c2c91.tar.gz* (if generated)